Course Name:	COMP 2233 Data Structures		
Lecturers:	Assoc. Prof. Dr. Ömer ÇETİN Research Assistant Giray KILIÇ Research Asistant Hatice ÇAĞLAYANLAR		
Lab. Number:	4	Lab. Date:	13.10.2026 / 16.10.2026
Objectives:	Exp. 1: Dynamic 2D sales array in C; create, delete, add, remove, and update elements; matrix multiplication for daily revenue and cost. Analyze operation costs.		

Course Learning Outcomes (CLO):		Relevancy (1-5)
CLO (In1)	Students will be able to identify useful data structures to represent specific types of information.	5
CLO (Sk3)	Students will be able to design and implement linear data structures.	5
CLO (Co2)	Students will be able to design and implement efficient solutions for searching and sorting operations.	1

Pre-Lab Assignment

Review the topics below before the session. Experiment 1 uses two-dimensional arrays and nested loops.

Experiment 1 Preparation (C)

- **2D Arrays: Declare, initialize, and traverse int a[ROWS][COLS] with nested loops; practice row and column sums.**
- **2D Arrays and Functions: The column size must be given in the parameter, e.g. void f(double m[][METRICS]).**
- **Dynamic 2D Arrays: Review malloc and free for int **sales (one row-pointer array plus one block per row).**

Matrix Multiplication and Weighted Averages

- **Rule: A (m × n) × B (n × p) = C (m × p), where C[i][j] = Σ A[i][k] × B[k][j].**
- **By hand, multiply the sales matrix [[10, 5], [12, 8], [8, 6]] by the unit-value matrix [[5, 2], [8, 4]]. The first row of the result is [90, 40].**
- **Review weighted sums and weighted averages: Σ (value × weight) / Σ weight.**
- **Prepare pseudocode and a flowchart for the experiment menu (options 0–7).**

Development Environment Setup

- Compile with gcc -std=c11 -Wall experiment_1.c -o experiment_1.
- Plan test cases: empty matrix, full matrix (30 days), invalid day or product numbers, and deleting then recreating the matrix.

Pre-Lab Self-Check Questions

- *Check 1: Why must every row be freed before the row-pointer array?*
- *Check 2: In A × B, why must the number of columns of A equal the number of rows of B?*
- *Check 3: When a day is removed, why is it enough to shift row pointers instead of copying every element?*
- *Check 4: In int a[3][3], what is the row-major position (starting from 0) of a[2][1] in memory?*

Laboratory Experiment

COMP 2233 / Data Structures Laboratory — Experiment 1 (C)

Topic: Multidimensional Arrays — Create / Delete a 2D Array, Element Operations, and Matrix Multiplication

1. Problem Description

A cafe sells two products: coffee and sandwiches. The units sold per day are stored in a sales matrix (rows: days, columns: products). The unit revenue and unit cost of each product are stored in a unit-value matrix (rows: products, columns: revenue and cost). Multiplying the two matrices gives the total revenue and total cost of each day.

Sales (days × 2) × Unit values (2 × 2) = Daily totals (days × 2)

2. Constraints and Specifications

- Sales matrix: int **sales, allocated with malloc; at most 30 days (MAX_DAYS); track the active day count.
- Sample data for three days: Day 1: 10 coffee, 5 sandwiches; Day 2: 12, 8; Day 3: 8, 6.
- Unit-value matrix: double unit_values[2][2]; coffee 5.00 revenue / 2.00 cost; sandwich 8.00 revenue / 4.00 cost.
- Input ranges: units 0–1000, unit values 0–1000. Day and product numbers start at 1.
- read_int and read_double are given in the template; they ask again for invalid input.

3. Experimental Tasks and Deliverables

Complete the empty functions in experiment_1_template.c:

1. Write pseudocode and draw a flowchart for the menu.
2. Create / Delete 2D Array:
    - create_matrix: allocate MAX_DAYS row pointers, allocate a row for each sample day, copy the sample sales, and set the day count to 3.
    - delete_matrix: free every active row, then the row-pointer array; main then sets sales to NULL.
3. Add / Remove / Update Elements:
    - add_day: if the matrix is not full, allocate a new row and read the coffee and sandwich units.
    - remove_day: free the selected row, shift the following row pointers up, and decrease the day count.
    - update_element: read a day and a product number and replace that single sales value.
    - update_unit_values: read new revenue and cost values for both products.
4. Matrix Multiplication:
    - print_totals: use three nested loops (day, metric, product) to compute Totals = Sales × Unit values.
    - Print the sales matrix, the unit-value matrix, and the revenue, cost, and profit (revenue − cost) of each day.
5. Complexity (D: days, P: products, M: metrics):
    - Update an element: O(1). Add a day: O(P). Remove a day and delete the matrix: O(D). Multiplication: O(D × P × M).

Sample Terminal I/O (Experiment 1):

```
Choice: 6
...
Daily totals = Sales x Unit values
Day 1: Revenue = 90.00, Cost = 40.00, Profit = 50.00
Day 2: Revenue = 124.00, Cost = 56.00, Profit = 68.00
Day 3: Revenue = 88.00, Cost = 40.00, Profit = 48.00
```

```
Choice: 4
Day number: 1
Product (1: coffee, 2: sandwich): 2
New number of units: 10
Sales element updated.
Day 1: Revenue = 10 × 5 + 5 × 8 = 90; Cost = 10 × 2 + 5 × 4 = 40. After the update, Day
1 revenue becomes 10 × 5 + 10 × 8 = 130.
```

Additional Knowledge (Supplementary Reference)

Key ideas:

1. **Static and Dynamic 2D Arrays**
   - **A static array such as int a[3][2] is one contiguous block stored row by row (row-major order).**
   - **Address Calculation:**

Address(i, j) = Base Address + (i × COLS + j) × sizeof(Element)

A dynamic int ** matrix is an array of row pointers; each row is a separate malloc block, so rows can be added or removed individually.

2. **Creating and Deleting a Dynamic 2D Array**
   - **Create: allocate the row-pointer array first, then each row; check malloc results for NULL.**
   - **Delete: free each row first, then the row-pointer array; then set the pointer to NULL to avoid a dangling pointer.**
   - **Freeing only the row-pointer array leaks the memory of every row.**
3. **Adding, Removing, and Updating Elements**
   - **Update: direct indexing, sales[i][j] = value, takes O(1).**
   - **Remove a row: free it and shift the later row pointers; only pointers move, not the data.**
   - **A static 2D array cannot shrink, so a removed element is usually marked (for example, set to 0).**
4. **Matrix Multiplication**
   - **A (m × n) × B (n × p) = C (m × p); the columns of A must equal the rows of B.**
   - **C[i][j] = Σ A[i][k] × B[k][j]; set C[i][j] to 0 before accumulating.**
   - **In general, A × B ≠ B × A.**
5. **Weighted Sums and Averages in 2D Arrays**
   - **Row-wise weighted sum: S[i] = Σ M[i][j] × w[j] (a matrix-vector product).**
   - **Weighted average = Σ (value × weight) / Σ weight; check that the divisor is not 0.**
6. **Time and Space Complexity (Big-O)**
   - **Element access and update: O(1). Traversing an R × C matrix: O(R × C).**
   - **Multiplication of (m × n) and (n × p) matrices: O(m × n × p). Storage: O(R × C).**

Attachments

A. C template: experiment_1_template.c; B. C solution: experiment_1.c

References

1. Weiss, M. A. (2013). Data Structures and Algorithm Analysis in C (2nd ed.). Pearson.

2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
3. Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
4. ISO/IEC 5807:1985. Information processing — Documentation symbols and conventions for data, program and system flowcharts.